

中山大学计算机院本科生实验报告

(2025 学年春季学期)

课程名称：并程序序设计

批改人：

实验	Pthreads 并行方程求解及蒙特卡洛	专业（方向）	计算机科学与技术
学号	22320021	姓名	陈安康
Email	1743826979@qq.com	完成日期	2025/4/22

1. 实验目的

利用 Pthreads 实现任务划分应用于一元二次方程求解。利用 Pthreads 实现数据划分应用于蒙特卡洛方法求 π 近似值。

2. 实验过程和核心代码

依据求根公式，将中间结果的计算分布到不同的线程中进行计算，并且通过条件变量进行线程之间的同步。核心代码如下：

```
typedef struct {
    double a, b, c;
    double b2, four_ac, discriminant, sqrt_discriminant, two_a;
    double x1, x2;

    // 状态标志
    int b2_ready, fourac_ready, discrim_ready, sqrt_ready, twoa_ready;

    // 锁与条件变量
    pthread_mutex_t mutex;
    pthread_cond_t cond;
} SharedData;

void wait_for(int* flag, SharedData* data) {
    while (!(*flag)) {
        pthread_cond_wait(&data->cond, &data->mutex);
    }
}
```

```

void* compute_x1(void* arg) {
    SharedData* data = (SharedData*) arg;
    pthread_mutex_lock(&data->mutex);
    wait_for(&data->sqr_ready, data);
    wait_for(&data->twoa_ready, data);

    if (data->sqr_discriminant < 0) {
        data->x1 = NAN;
    } else {
        data->x1 = (-data->b + data->sqr_discriminant) / data->two_a;
    }
    pthread_mutex_unlock(&data->mutex);
    return NULL;
}

```

```

// 创建线程
pthread_t threads[7];
pthread_create(&threads[0], NULL, compute_b2, &data);
pthread_create(&threads[1], NULL, compute_4ac, &data);
pthread_create(&threads[2], NULL, compute_discriminant, &data);
pthread_create(&threads[3], NULL, compute_sqrt, &data);
pthread_create(&threads[4], NULL, compute_2a, &data);
pthread_create(&threads[5], NULL, compute_x1, &data);
pthread_create(&threads[6], NULL, compute_x2, &data);

// 等待结果线程完成
pthread_join(threads[5], NULL);
pthread_join(threads[6], NULL);

// 结束计时
clock_gettime(CLOCK_MONOTONIC, &end);
double elapsed_sec = end.tv_sec - start.tv_sec + (end.tv_nsec - start.tv_nsec) / 1e9;

if (isnan(data.x1)) {
    printf("无实数根。\\n");
} else {
    printf("x1 = %.9f, x2 = %.9f\\n", data.x1, data.x2);
}

printf("总耗时: %.9f 秒\\n", elapsed_sec);

pthread_mutex_destroy(&data.mutex);
pthread_cond_destroy(&data.cond);
return 0;
}

```

作为参照，实现了串行版本。

蒙特卡洛方法求 π 近似值

思路是将随机数据点的生成分配给不同的线程并行执行，提高执行效率，将生成结果放在一个共享内存中，最后由主线程集合各个线程的结果得到最终的 π 估计值。核心代码如下：

```
// 线程执行函数
void* monte_carlo_pi(void* arg) {
    int tid = *(int*)arg;
    unsigned int seed = tid + time(NULL); // 每个线程使用不同种子
    long local_count = 0;

    long points_per_thread = total_points / num_threads;
    for (long i = 0; i < points_per_thread; ++i) {
        double x = rand_r(&seed) / (double)RAND_MAX;
        double y = rand_r(&seed) / (double)RAND_MAX;
        if (x * x + y * y <= 1.0) {
            local_count++;
        }
    }

    points_in_circle[tid] = local_count;
    pthread_exit(NULL);
}
```

作为参照，同时实现了串行版本。

3. 实验结果

由于在执行时发现并行和串行结果差距很大，串行结果性能远高于并行，因此在代码中添加一些统计代码。

任务 1 并行版本运行结果如下：

```

cake@CAK:~/并行程序设计/hw5$ ./yiyuanerci
请输入 a b c:
9 6 1
x1 = -0.333333333, x2 = -0.333333333
总耗时: 0.000645524 秒

线程计算和等待时间统计:
线程          计算时间(秒)    等待时间(秒)    总时间(秒)
计算b²         0.000000300      0.000000000      0.000000300
计算4ac        0.000000085      0.000000000      0.000000085
计算判别式     0.000000024      0.000000000      0.000000024
计算平方根     0.000000507      0.000000000      0.000000507
计算2a         0.000000056      0.000000000      0.000000056
计算x₁         0.000000069      0.000000000      0.000000069
计算x₂         0.000000043      0.000000000      0.000000043

总计算时间: 0.000001084 秒
总等待时间: 0.000000000 秒
计算/总时间比例: 0.17%
等待/总时间比例: 0.00%

线程创建开销统计:
总线程创建时间: 0.000185657 秒
线程创建/总时间比例: 28.76%

其他系统开销时间: 0.000458783 秒
其他系统开销/总时间比例: 71.07%

```

串行结果如下:

```

cake@CAK:~/并行程序设计/hw5$ ./yiyuanerci_serial
请输入 a b c (范围 [-100, 100]) :
9 6 1
x1 = -0.333333333, x2 = -0.333333333
总耗时: 0.000000967 秒

```

可以发现，串行结果性能远高于并行，在细致比较会发现，两者在纯计算上的开销差距并不大，但并行计算中线程创建，切换等时间开销占据了绝大部分。导致并行计算的性能反而低于串行。

蒙特卡洛方法求 π 近似值

首先运行串行版本，作为参考，结果如下：


```
● cake@CAK:~/并行程序设计/hw5$ ./montecarlo_pi_s 999999999
总点数: 999999999
圆内点数: 785394717
 $\pi$  的估计值: 3.141579
耗时: 10.671999 秒
```

接下来运行并行版本，不同线程数下的运行结果如下：

```
● cake@CAK:~/并行程序设计/hw5$ ./montecarlo_pi 2 999999999
总点数: 999999999
圆内点数: 785397071
 $\pi$  的估计值: 3.141588
耗时: 5.697497 秒
```

```
● cake@CAK:~/并行程序设计/hw5$ ./montecarlo_pi 4 999999999
总点数: 999999999
圆内点数: 785397481
 $\pi$  的估计值: 3.141590
耗时: 3.030869 秒
```

```
● cake@CAK:~/并行程序设计/hw5$ ./montecarlo_pi 8 999999999
总点数: 999999999
圆内点数: 785397562
 $\pi$  的估计值: 3.141590
耗时: 1.772216 秒
```

```
● cake@CAK:~/并行程序设计/hw5$ ./montecarlo_pi 16 999999999
总点数: 999999999
圆内点数: 785398700
 $\pi$  的估计值: 3.141595
耗时: 1.087539 秒
```

观察结果可以很明显的发现数据并行提升了运行效率，抛去线程开销，基本上可以做到线程数提升一倍，性能也相应提升一倍。由于我的 wsl 只有 16 个核，继续增加线程，结果如下：

```
● cake@CAK:~/并行程序设计/hw5$ ./montecarlo_pi 32 999999999
总点数: 999999999
圆内点数: 785401519
 $\pi$  的估计值: 3.141606
耗时: 1.050604 秒
```

会发现性能没有提升，这是因为线程数多于核数，并行转并发了。

检查实验结果，发现结果基本都可以精确到小数点后 4-5 位，计算结果有

效。

4. 实验感想

通过这次实验，我实现了一元二次方程的并行求解程序，并且同串行版本进行对比，分析了串行版本性能反而高于并行版本的原因，同时体会到在小规模问题时线程开销可能会“喧宾夺主”，导致并行版本性能降低。同时我实现了蒙特卡洛方法求解 π 近似值的数据划分的并行算法，在大规模问题时，并行相比于串行的性能提升还是很高的。通过此次实验，我体会到并行领域中任务划分和数据划分的区别，同时还提高了对并行选择的进一步认识，受益匪浅。